

Laboratorio 6: Transfer Learning y Fine-Tuning

Juan Francisco Martínez | Carnet 23617

Repositorio: <https://github.com/Cisco890/tarea6>

CIFAR-10 · CNN desde cero · VGG-16 feature extractor · VGG-16 fine-tuning (112×112)

Hardware: NVIDIA Tesla T4 (CUDA) — entorno de entrenamiento. Semilla 42. Split estratificado 45,000 / 5,000; test oficial de 10,000 imágenes.

1. Investigación: transfer learning en PyTorch

`torchvision.models.vgg16` y `VGG16_Weights.DEFAULT` construyen la red e inicializan pesos de ImageNet. `weights.transforms()` entrega el pipeline oficial (resize a 224×224 y normalización ImageNet); aquí se sustituye por un pipeline propio a 112×112 por costo computacional, manteniendo la misma resolución en feature extraction y fine-tuning.

La arquitectura se divide en `model.features` (bloques convolucionales), `model.avgpool` (*AdaptiveAvgPool2d*, tamaño espacial fijo) y `model.classifier` (MLP; la última *Linear* se reemplaza por 10 salidas). Con `requires_grad` se congelan o descongelan bloques. `train()/eval()` controlan Dropout y BatchNorm; `torch.no_grad()` evita guardar el grafo en inferencia. En fine-tuning se usan `param_groups` del optimizador para asignar un learning rate menor al backbone que al clasificador.

Para costo computacional se usa `fvcore.FlopCountAnalysis` (FLOPs del forward). En GPU, `torch.cuda.max_memory_allocated` y `synchronize` permiten reportar memoria pico y tiempos fiables. Un MAC $\approx$ 2 FLOPs; se mantiene la misma convención entre modelos. Los parámetros entrenables son el subconjunto con `requires_grad=True`.

En datos: CIFAR-10 está balanceado (5,000 imgs/clase en train). Media/std por canal calculadas en train: R=0.491±0.247, G=0.482±0.243, B=0.447±0.262 (ImageNet usa 0.485/0.456/0.406 y 0.229/0.224/0.225). La CNN normaliza con CIFAR; VGG con ImageNet. Augmentation solo en train. Pasar de 32×32 a 112×112 multiplica píxeles por $\approx$ 12.25× (224×224 sería 49×).

2. Resultados de las 9 iteraciones (validación)

Se ejecutaron tres configuraciones por familia. Selección del mejor modelo **únicamente por F1 macro de validación** (sin mirar test). Early stopping con paciencia 3. CNN en 32×32; VGG en 112×112.

experiment	unfrozen_blocks	best_epoch	best_val_f1	best_val_accuracy	trainable_params	total_training_time_s
CNN-A	-	8	0.7436	0.7463	323498	126
CNN-C	-	8	0.7178	0.7276	323498	138
CNN-B	-	8	0.7049	0.7115	323498	129
VGG-FE-B	none	6	0.7792	0.7869	119586826	340
VGG-FE-A	none	6	0.7646	0.7718	119586826	327
VGG-FE-C	none	6	0.7594	0.7624	119586826	325
VGG-FT-B	4+5	7	0.8489	0.8512	132566026	610
VGG-FT-A	5	7	0.8254	0.8356	126666250	582
VGG-FT-C	5	7	0.8242	0.8270	126666250	593

Mejores por familia: **CNN-A** (LR=1e-3, dropout=0.3), **VGG-FE-B** (LR clasificador=3e-4) y **VGG-FT-B** (bloques 4+5, LR backbone=5e-6, clasificador=1e-4). En fine-tuning, descongelar 4+5 superó a solo el bloque 5.

3. Comparación de rendimiento y recursos

3.1 Métricas de test y experimento con 10 % de datos

Modelo	Test Acc	Test F1	F1 10%	Caída abs.	Caída rel.
CNN	0.7320	0.7325	0.6191	0.1134	15.5%
VGG-FE	0.7750	0.7756	0.7029	0.0726	9.4%
VGG-FT	0.8420	0.8423	0.7943	0.0480	5.7%

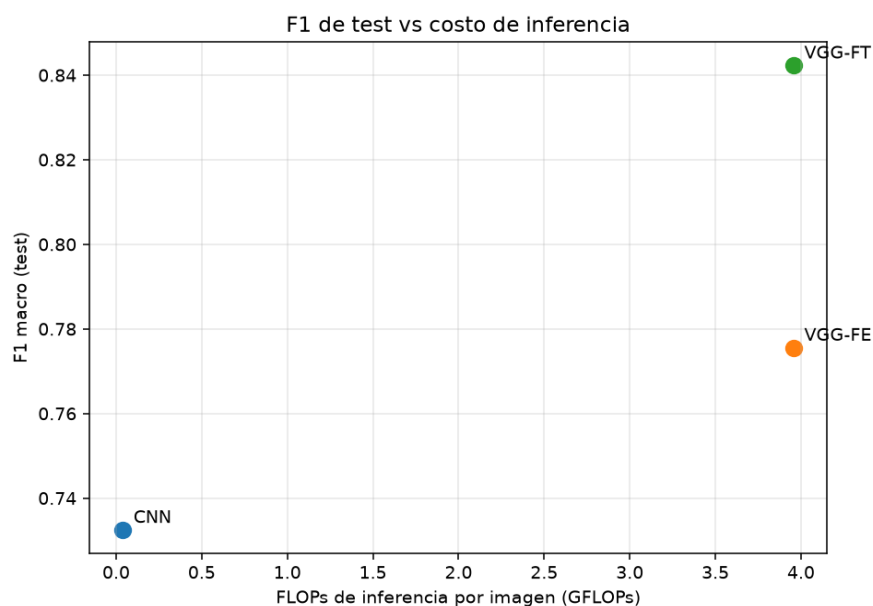
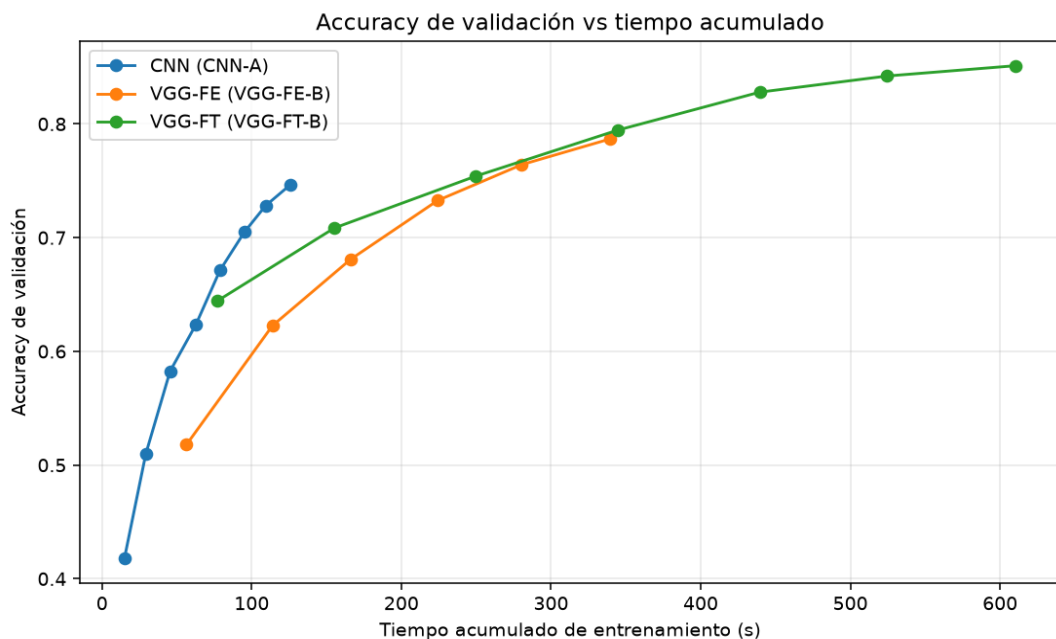
3.2 Recursos de los mejores modelos

family	test_f1	f1_10%	params	trainable	FLOPs/img	lat. ms	VRAM MB
CNN	0.7325	0.6191	323,498	323,498	0.039G	0.35	845
VGG-FE	0.7756	0.7029	134,301,514	119,586,826	3.956G	4.85	2115

family	test_f1	f1_10%	params	trainable	FLOPs/img	lat. ms	VRAM MB
VGG-FT	0.8423	0.7943	134,301,514	132,566,026	3.956G	5.12	3560

La estimación de FLOPs de entrenamiento usa $FLOPs_{forward} \times \#imágenes \times epochs_{hasta_mejor} \times factor$, con factor ≈ 3 (CNN), ≈ 1.15 (FE, backward solo en clasificador) y ≈ 2.2 (FT con bloques 4+5). Es una aproximación documentada, no una medición exacta del autograd.

3.3 Gráficas comparativas



4. Análisis y conclusiones

Rendimiento final y trade-off

El mejor F1 de test es **VGG-FT** (0.8423, accuracy 0.8420). Respecto a feature extraction gana +0.0667 F1, y frente a la CNN +0.1098. Ese margen justifica el costo extra en entrenamiento (≈ 610 s vs ≈ 340 s en FE y ≈ 126 s en CNN) y en memoria pico (3560 MB). En inferencia, FE y FT tienen FLOPs casi idénticos (~ 3.96 GFLOPs); la CNN es dos órdenes de magnitud más ligera (38.9 MFLOPs, latencia ~ 0.35 ms).

Features preentrenadas y redimensionamiento

Aunque VGG-16 se preentrenó a 224×224 e ImageNet y CIFAR-10 es 32×32 , los filtros tempranos (bordes, texturas, color) siguen siendo útiles. Redimensionar a 112×112 no crea detalle nuevo, pero alinea mejor la escala de activaciones; el costo es

≈12.25× más píxeles que la resolución nativa. Usar 224×224 habría multiplicado el cómputo por ~49× respecto a 32×32, inviables bajo la ventana de este laboratorio.

Fine-tuning, overfitting y datos reducidos

Descongelar bloques 4+5 (VGG-FT-B) mejoró el F1 de validación frente a solo el bloque 5, con learning rates bajos en el backbone (5e-6) para limitar *catastrophic forgetting*. Las curvas muestran convergencia estable sin colapso de validación. En el experimento al 10 % de train, la CNN pierde 0.113 F1 absoluto, FE 0.073 y FT 0.048: con pocos datos conviene transfer learning; con datos abundantes una CNN nativa sigue siendo atractiva por costo.

Errores por clase y despliegue

Las matrices de confusión concentran errores en pares visualmente ambiguos a baja resolución: *cat↔dog*, *automobile↔truck*, *bird↔airplane* y *deer↔horse*. El fine-tuning reduce estas confusiones pero no las elimina. Para despliegue en **servidor con GPU** se recomienda VGG-FT (mejor F1). En **dispositivo móvil**, la CNN propia o, preferiblemente, un backbone ligero como **MobileNetV3** o **EfficientNet-B0**, con muchos menos parámetros y FLOPs que VGG-16.